AutoInsight — System Architecture

Automated Prompt-Driven Data Analysis and Dashboard Generation System with Traceable Insights

1. USER INTERFACE — Lovable-generated React / TypeScript

Excel upload • Natural-language prompt • Data preview • Dashboard • Ranked insights • Traceability • Follow-up Q&A

↓ HTTP requests

2. PYTHON BACKEND API — FastAPI

Validates requests and uploaded files • Coordinates workflow • Returns structured results to the frontend

↓ dispatches tasks

3. MULTI-AGENT ORCHESTRATION

Coordinator Agent: interprets the prompt and delegates tasks

Data Agent	Analysis Agent	Visualization Agent	Validation Agent
Column types, data quality, cleaning and transformation logs	Statistics, trends, correlations, forecasting and supported analysis	Chart selection and dashboard preparation	Calculation checks, evidence and result consistency

↓ Python functions first; optional local LLM for flexible language tasks

4. ANALYSIS & INTELLIGENCE ENGINE

Pandas / NumPy for processing • SciPy / scikit-learn / statsmodels for suitable methods • Plotly for charts • Rules or local LLM for narrative insights • Evidence and traceability records

↓ stores outputs and prepares dashboard data

5. STORAGE & DASHBOARD INTEGRATION

Initial version: local temporary files and structured logs • Store analysis results, cleaning history and visualization history • Tableau template workbook + Hyper extract workflow only if supported by available access and licensing

↓ returns results

6. USER OUTPUT

Interactive charts and dashboards • Plain-English insights • Confidence indicators where methodologically justified • Forecast metrics (e.g., MAPE / RMSE) • Fairness flags • Source-row / formula traceability • Dataset-grounded Q&A

Design principle: start with a working single workflow and Python modules; introduce LLM-powered agents and advanced integrations incrementally. Target: no paid AI API spending. Free-tier limits and Tableau capabilities must be verified before implementation.

Implementation Roadmap

Build and test one milestone at a time. These are stages, not fixed deadlines.

Stage	Work	Completion check
1. Website prototype	Create the mobile-friendly interface in Lovable; connect GitHub.	Website screens work and code is stored in the repository.
2. Local development	Clone repository to PC; open in VS Code and configure Cline.	Project runs locally and changes can be committed.
3. Excel upload	Build FastAPI endpoint and read Excel files with Pandas.	Upload a file and display its columns and rows.
4. Basic analysis	Add cleaning, descriptive statistics and one chart.	Real uploaded data produces reproducible results.
5. Agent coordination	Separate data, analysis, visualization and validation tasks; add coordinator.	Tasks pass structured results and failures are handled.
6. Trust features	Add traceability, validated insights, suitable forecasting and fairness checks.	Findings link to evidence and metrics are calculated correctly.
7. Q&A and Tableau	Add dataset-grounded questions and test Tableau integration if feasible.	Answers use dataset evidence; dashboard workflow is demonstrated.
8. Final testing	Test multiple files, handle errors, document and prepare demo.	End-to-end demonstration and documentation are ready.

Technology Stack

Layer	Proposed technology
Frontend	Lovable-generated React / TypeScript
Version control / development	GitHub • VS Code • Cline
Backend	Python • FastAPI
Data and analytics	Pandas • NumPy • SciPy • scikit-learn • statsmodels
Charts	Plotly
LLM option	Local model via Ollama or compatible runner, if suitable; not required for the first milestone
Dashboard integration	Tableau and Hyper extract workflow, subject to access and licensing

Note: This document describes the proposed architecture, not an already implemented system. Initial target: zero paid AI API spending; feature availability, free-tier limits and Tableau licensing must be checked.